
Verification Plan for 8-bit ALU

Submitted to:

Eng. Mohamed El-Adawy

Submitted by:

Menna Assem Fouda

1. Block Interface and Signal Specifications

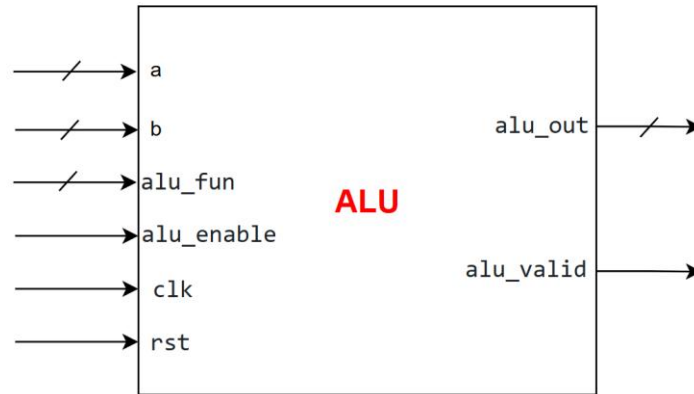


Figure 1:ALU Block Interface

Port	Width	Direction	Description
a	Parameterized (8 bits)	IN	Operand 1
b	Parameterized (8 bits)	IN	Operand 2
alu_fun	2 bits	IN	Function selection
alu_enable	1 bit	IN	Enable signal
alu_clk	1 bit	IN	Clock signal
alu_rst	1 bit	IN	Active low reset
alu_out	Parameterized (8 bits)	OUT	Alu result
alu_valid	1 bit	OUT	Valid output signal

- All registers are cleared using Asynchronous active low reset
- **Alu_valid** is High when ALU result is ready, otherwise LOW.
- The ALU function is carried out according to the value of the **alu_fun** input signal stated in the following table
- Results are regestered only if **alu_enable** HIGH.

2. Verification Plan

1] **rst** → "All output signals must be zero"

alu_out = 0

2] **a, b (8-bit inputs)** → Valid or Invalid

Cover edge values: 0x00, 0xFF, 0x01, max/min combinations, random

3] **alu_fun (2-bit)** → Function select — encoding:

00 → ADD (a + b)

01 → SUB ($a - b$)

10 → AND ($a \& b$)

11 → OR ($a | b$)

Note: all 4 encodings of alu_fun are legal — there is no invalid/reserved encoding for this 2-bit field.

4] alu_enable → @posedge clk, controls whether alu_out updates

Set to 1 → ALU executes the operation selected by alu_fun and alu_out updates on the next clock edge.

Set to 0 → alu_out holds its last value (no update).

5] alu_out → Checker (registered output, 1-cycle delay from inputs)

If reset asserted → check $\text{alu_out} == 0$

@posedge clk: if alu_enable was high on the previous cycle → check $\text{alu_out} == (a \text{ op } b)$ per alu_fun, using that previous cycle's a, b, alu_fun

If alu_enable was low on the previous cycle → check alu_out holds its last value

6] Enable Low → Checker

If alu_enable is low → check alu_out holds its last value regardless of changes to a, b, or alu_fun.

alu_valid is unaffected by alu_enable (it is purely combinational on alu_fun) and should continue to track alu_fun normally even while alu_enable is low.

7] alu_valid → Cross-check

alu_valid must read 1 for every legal value of alu_fun (ADD, SUB, AND, OR) — i.e. for all 4 encodings of this 2-bit field, since none are reserved/invalid.

3. Waveform snippets

We need to cover all output signals:

- alu_out → cover all values (0x00 – 0xFF)
- alu_out \ ADD → cover output wraparound past 0xFF (e.g. $0xFF + 0x01 \rightarrow 0x00$)
- alu_out \ SUB → cover output wraparound below 0x00 (e.g. $0x00 - 0x01 \rightarrow 0xFF$)
- alu_valid → cover $\text{alu_valid} == 1$ for each of the 4 alu_fun values (ADD, SUB, AND, OR)
- alu_fun → cover all 4 function values (ADD, SUB, AND, OR)
- alu_enable → cover operation using $\text{alu_enable} == 1$
- alu_enable → cover hold state using $\text{alu_enable} == 0$
- a, b → cover cross of operand corner values (0x00, 0xFF, mid-range) × all alu_fun values

4. Sequences & Constrained Randomization

1] Reset Sequence → with $\text{rst} == 0$

2] Enable / Disable Sequence → with $\text{alu_enable} == 1 / 0$

3] ADD Sequence → with $\text{alu_fun} == 00$, a and b random

4] SUB Sequence → with `alu_fun == 01`, a and b random

5] AND Sequence → with `alu_fun == 10`, a and b random

6] OR Sequence → with `alu_fun == 11`, a and b random

7] Output-Wraparound Sequence → with `alu_fun == 00` (ADD), a and b constrained such that $a + b > 0xFF$ (forces output wraparound)

with `alu_fun == 01` (SUB), a and b constrained such that $a < b$ (forces output wraparound)

5. Test Scenarios

Scenario 1: ADD Sequence for random operand transactions

Goal: testing basic arithmetic functionality after reset

Scenario 2: SUB Sequence for random operand transactions

Goal: testing subtraction functionality after reset

Scenario 3: ADD → AND → SUB → OR → ADD ... etc. (random function switching each cycle)

Goal: mixing the order of operations, checking `alu_out` and `alu_valid` update correctly on every function change

Scenario 4: Enable/Disable Sequence → random for at least 10 times

Goal: testing `alu_enable` gating — verifying `alu_out` holds when disabled and updates correctly when re-enabled

Scenario 5: ADD and SUB with operands forcing output wraparound, repeated for at least 10 times

Goal: testing `alu_out` accuracy at boundary conditions (0x00, 0xFF)

Scenario 6: Reset asserted mid-operation, random timing, repeated for at least 10 times

Goal: testing that asynchronous reset overrides any in-progress operation, forcing `alu_out` to zero